A 小苯的选择题

- 4 分：相等
- 2 分：子序列（因为保证是升序给出）
- 0 分：剩余情况

```
bool subsequence(string s, string t) {
    int i = 0;
    for (char c : t) {
        if (i < s.size() && s[i] == c) {
            i++;
        }
    }
    return i == s.size();
}

void solve() {
    string s, tar = "ABD";
    cin >> s;
    if (s == tar) {
        cout << 4 << endl;
    } else if (subsequence(s, tar)) {
        cout << 2 << endl;
    } else {
        cout << 0 << endl;
    }
}
```

Fence 1

B 小苯的峰值序列

不难想到：

- 首尾是不可动的。
- 对于一个峰 $a[i]$ ，我们最多可以把它减少到 $max(a[i-1], a[i+1])$ ，并且把相邻两位锁住。

那么只需要对每个峰单独考虑即可。

```
void solve() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }
    for (int i = 1; i < n - 1; ++i) {
        if (a[i] > a[i - 1] && a[i] > a[i + 1]) {
            a[i] = max(a[i - 1], a[i + 1]);
        }
    }
    for (auto v : a) {
        cout << v << " ";
    }
    cout << endl;
}
```

C 小苯排雷

因为会连锁反应把相邻的炸弹全引爆，所以我们把连续的炸弹归成一组，组的花费就是组内炸弹最小花费。

```cpp
void solve() {
    int n;
    cin >> n;
    vector<ll> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }
    ll ans = 0, mn = INF;
    for (int i = 0; i < n; ++i) {
        if (a[i]) {
            mn = min(mn, a[i]);
        } else {
            if (mn != INF) {
                ans += mn;
            }
            mn = INF;
        }
    }
    if (mn != INF) {
        ans += mn;
    }
    cout << ans << endl;
}
```

D 小苯的01合并

首先如果 A 的异或和不等于 B 的异或和，A 肯定不能变成 B 。

然后我们贪心模拟 A 的变化即可，因为提前切割可以给后面留下更多空间，并且异或只有 $0/1$，不存在更优的延后收益。

```
void solve() {
    int n, m;
    cin >> n >> m;
    string A, B;
    cin >> A >> B;
    int a = 0, b = 0;
    for (auto v : A) {
        a ^= v - '0';
    }
    for (auto v : B) {
        b ^= v - '0';
    }
    if (a != b) {
        cout << "NO" << endl;
        return;
    }
    int pos = 0;
    bool ok = true;

    for (int i = 0; i < m - 1; i++) {
        int cur = 0;
        bool f = false;

        for (int j = pos; j <= n - (m - i - 1) - 1; j++) {
            cur ^= (A[j] - '0');
            if (cur == (B[i] - '0')) {
                pos = j + 1;
                f = true;
                break;
            }
        }

        if (!f) {
            ok = false;
            break;
        }
    }

    if (!ok) {
        cout << "NO" << endl;
        return;
    }

    int cur = 0;
    for (int i = pos; i < n; i++) {
        cur ^= (A[i] - '0');
    }
```

```

    if (cur == (B[m - 1] - '0')) {
        cout << "YES" << endl;
    } else {
        cout << "NO" << endl;
    }
}
```

Fence 4

E 小苯的数组操作

法0（std）：

操作 1 总是覆盖前缀，操作 2 总是覆盖后缀。
我们维护两个指针 L 和 R，以及两个值 s1 和 s2：

- L：表示前缀 $[1, L]$ 已经被操作 1 推平成了同一个值 s1。
- R：表示后缀 $[R, n]$ 已经被操作 2 推平成了同一个值 s2。
- 中间部分 (L, R)：保持原数组的值不变（通过前缀和 s 快速计算）。

当操作发生时：

1. 前缀操作：更新 $L = max(L, i)$。新的 $s1$ 取当前 $s1$ 和原数组前缀最小值 $pre[L]$ 的较小值。
2. 后缀操作：更新 $R = min(R, i)$。新的 $s2$ 取当前 $s2$ 和原数组后缀最小值 $suf[R]$ 的较小值。
3. 重叠：如果 L 和 R 发生重叠（即 $L \geq R$），说明当前操作的区间与之前的区间联合覆盖了整个数组，因此当前操作的区间的值会变为全局最小值。

```cpp
void solve() {
    int n, q;
    cin >> n >> q;

    vector<ll> a(n + 1), s(n + 1, 0), pre(n + 2, INF), suf(n + 2, INF);

    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        s[i] = s[i - 1] + a[i];
        pre[i] = min(pre[i - 1], a[i]);
    }

    for (int i = n; i >= 1; i--) {
        suf[i] = min(suf[i + 1], a[i]);
    }

    int L = 1, R = n;
    ll s1 = a[1], s2 = a[n];

    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int i;
            cin >> i;
            L = max(L, i);

            if (L < R) {
                s1 = min(s1, pre[L]);
            } else {
                s1 = pre[n];
                R = L + 1;
            }
        } else if (op == 2) {
            int i;
```

```
            cin >> i;
            R = min(R, i);

            if (L < R) {
                s2 = min(s2, suf[R]);
            } else {
                s2 = pre[n];
                L = R - 1;
            }
        } else {
            ll ans = s1 * L + s2 * (n - R + 1) + (s[R - 1] - s[L]);
            cout << ans << endl;
        }
    }
}
```

Fence 5

数据结构解法：

- 法1：

我只用了 0 秒就想到使用区间赋值线段树，你也来试试吧。

```
void solve() {
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
    }
    SegTree<int> seg(n);
    seg.build(a);
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int i;
            cin >> i;
            int mn = seg.getMin(1, i);
            seg.set(1, i, mn);
        } else if (op == 2) {
            int i;
            cin >> i;
            int mn = seg.getMin(i, n);
            seg.set(i, n, mn);
        } else {
            cout << seg.getSum(1, n) << endl;
        }
    }
}
```

Fence 6

法2：

我们还可以使用珂朵莉树的思想，题目中的两个修改操作具有区间推平（将一段区间赋值为同一个数）的特性：

- 操作 1 将 $[1, i]$ 变为这段区间的最小值，操作后 $[1, i]$ 变成了一个值相同的块。
- 操作 2 将 $[i, n]$ 变为这段区间的最小值，操作后 $[i, n]$ 变成了一个值相同的块。

这种“区间赋值”的操作非常适合用 set 来维护若干个形如 $[l, r, val]$ 的区间。

- 每次操作时，我们只需要把涉及到的区间找出来。
- 遍历这些区间，算出最小值，并从总和 sum 中减去这些区间的贡献。
- 删除这些区间，然后插入一个新的合并后的区间，并加上新值的贡献。

```
void solve() {
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
    }

    ODT<ll> tree(n);
    tree.build(a);

    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int i;
            cin >> i;
            tree.setMin(1, i);
        } else if (op == 2) {
            int i;
            cin >> i;
            tree.setMin(i, n);
        } else {
            cout << tree.sum << endl;
        }
    }
}
```

F 小苯的数字分块

一个数组的“块数”可以通过统计相邻元素的变化来计算。对于 i 从 0 到 $n-2$（即相邻对 a_i, a_{i+1} ），如果 $a_i \neq a_{i+1}$ ，则说明 a_{i+1} 开启了一个新的块。

根据期望的线性性质：

$$E[B] = 1 + \sum_{i=0}^{n-2} P(a_i \neq a_{i+1})$$

我们只需要计算每一对相邻元素不相等的概率 $P(a_i \neq a_{i+1})$ ，然后将它们累加即可。

直接计算不相等的概率比较麻烦，我们可以先计算相等的概率 $P(a_i = ai + 1)$ ，然后用 1 减去它。

$a_i = a_{i+1}$ 的情况数等于两个区间的交集大小，总状态数为两个区间大小的积。

```
void solve() {
    int n;
    cin >> n;
    vector<ll> l(n), r(n);
    for (int i = 0; i < n; ++i) {
        cin >> l[i] >> r[i];
    }
    Z ans = 1;
    for (int i = 0; i < n - 1; ++i) {
        Z l1 = r[i] - l[i] + 1, l2 = r[i + 1] - l[i + 1] + 1;

        ll L = max(l[i], l[i + 1]), R = min(r[i], r[i + 1]);

        Z over = 0;
        if (L <= R) {
            over = R - L + 1;
        }

        ans += 1 - over / (l1 * l2);
    }

    cout << ans << endl;
}
```

头文件

```cpp
#include<bits/stdc++.h>
#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define all(a) a.begin(), a.end()
#define rall(a) a.rbegin(), a.rend()
#define endl "\n"
using namespace std;

using ll = long long;
using ld = long double;
using ull = unsigned long long;
using i128 = __int128;
using PII = pair<int, int>;
using PLL = pair<ll, ll>;
using TIII = tuple<int, int, int>;
using TLLL = tuple<ll, ll, ll>;

constexpr int inf = (ll)1e9 + 7;
constexpr ll INF = (ll)2e18 + 9;
// constexpr ll INF = (ll)4e18;
// constexpr ll MOD = 1e9 + 7;
constexpr ll MOD = 998244353;
constexpr ld eps = 1e-10;

void init() {

}

void solve() {

}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    init();

    int t = 1;
    cin >> t;
    for (int _ = 1; _ <= t; ++_) {
        solve();
    }

    return 0;
}
```

线段树

```cpp
template <class Int> struct Tag {
    Int add = 0;
    Int set = 0;
    bool f = false;

    static Tag make_add(Int v) { return {v, 0, false}; }
    static Tag make_set(Int v) { return {0, v, true}; }

    void operator+=(const Tag<Int> &o) {
        if (o.f) {
            f = true;
            set = o.set;
            add = 0;
        } else {
            if (f) {
                set += o.add;
            } else {
                add += o.add;
            }
        }
    }

    bool check() { return f || add != 0; }
};

template <class Int> struct Info {
    Int sum = 0;
    Int mx = 0;
    Int mn = 0;
    int l = 0, r = 0;

    void set(int pos, Int val) {
        l = r = pos;
        sum = val;
        mx = val;
        mn = val;
    }

    Info operator+(const Info<Int> &o) const {
        if (l == 0)
            return o;
        if (o.l == 0)
            return *this;

        Info res;
        res.l = l;
        res.r = o.r;
        res.sum = sum + o.sum;
        res.mx = max(mx, o.mx);
        res.mn = min(mn, o.mn);
        return res;
    }
```

```cpp

    void operator+=(const Tag<Int> &o) {
        if (o.f) {
            sum = o.set * (r - l + 1);
            mx = o.set;
            mn = o.set;
        } else {
            sum += o.add * (r - l + 1);
            mx += o.add;
            mn += o.add;
        }
    }

    bool check() { return l != r; }
};

template <class Int> class SegTree {
  private:
    vector<Info<Int>> info;
    vector<Tag<Int>> tag;
    int n;

    int ls(int x) { return x << 1; }
    int rs(int x) { return x << 1 | 1; }

    void print(int x, int l, int r) {
        if (l == 0)
            return;
        cout << x << ":[" << l << "," << r << "] sum:" << info[x].sum
             << " max:" << info[x].mx << " tag_add:" << tag[x].add
             << " tag_set:" << tag[x].set << " f:" << tag[x].f << "\n";
        if (l == r)
            return;
        int mid = (l + r) >> 1;
        print(ls(x), l, mid);
        print(rs(x), mid + 1, r);
    }

    template <class Array> void build(int x, int l, int r, Array &data) {
        if (l == r) {
            info[x].set(l, data[l]);
            return;
        }
        int mid = (l + r) >> 1;
        build(ls(x), l, mid, data);
        build(rs(x), mid + 1, r, data);
        info[x] = info[ls(x)] + info[rs(x)];
    }

    void push_down(int x) {
        if (tag[x].check()) {
            info[ls(x)] += tag[x];
            info[rs(x)] += tag[x];
            tag[ls(x)] += tag[x];
            tag[rs(x)] += tag[x];
            tag[x] = {0, 0, false};
```

```
        }
    }

    void update(int x, int l, int r, int lq, int rq, Tag<Int> v) {
        if (rq < l || lq > r)
            return;
        if (lq <= l && r <= rq) {
            info[x] += v;
            tag[x] += v;
            return;
        }
        push_down(x);
        int mid = (l + r) >> 1;
        update(ls(x), l, mid, lq, rq, v);
        update(rs(x), mid + 1, r, lq, rq, v);
        info[x] = info[ls(x)] + info[rs(x)];
    }

    Info<Int> ask(int x, int l, int r, int lq, int rq) {
        if (rq < l || lq > r)
            return Info<Int>();
        if (lq <= l && r <= rq)
            return info[x];
        push_down(x);
        int mid = (l + r) >> 1;
        return ask(ls(x), l, mid, lq, rq) + ask(rs(x), mid + 1, r, lq, rq);
    }

  public:
    SegTree(int n_) : n(n_), info(4 * n_ + 1), tag(4 * n_ + 1) {}

    template <class Array> void build(Array &data) { build(1, 1, n, data); }

    void add(int l, int r, Int v) {
        update(1, 1, n, l, r, Tag<Int>::make_add(v));
    }

    void set(int l, int r, Int v) {
        update(1, 1, n, l, r, Tag<Int>::make_set(v));
    }

    void modify(int pos, Int v) { set(pos, pos, v); }

    Int getSum(int l, int r) { return ask(1, 1, n, l, r).sum; }

    Int getMax(int l, int r) { return ask(1, 1, n, l, r).mx; }

    Int getMin(int l, int r) { return ask(1, 1, n, l, r).mn; }

    Info<Int> query_info(int l, int r) { return ask(1, 1, n, l, r); }

    void print() { print(1, 1, n); }
};
```

珂朵莉树

```
template <class Int> struct Node {
    int l, r;
    mutable Int v;
    bool operator<(const Node &o) const { return l < o.l; }
};

template <class Int> struct ODT {
    set<Node<Int>> s;
    Int sum = 0;
    int n = 0;

    ODT(int n_) : n(n_) {}

    template <class Array> void build(Array &data) {
        s.clear();
        sum = 0;
        for (int i = 1; i <= n;) {
            int j = i;
            while (j <= n && data[j] == data[i])
                j++;
            s.insert({i, j - 1, data[i]});
            sum += data[i] * (j - i);
            i = j;
        }
    }

    auto split(int pos) {
        if (pos > n)
            return s.end();
        auto it = s.lower_bound({pos, 0, Int()});
        if (it != s.end() && it->l == pos)
            return it;
        if (it == s.begin())
            return s.end();
        --it;
        int l = it->l, r = it->r;
        Int v = it->v;
        s.erase(it);
        s.insert({l, pos - 1, v});
        return s.insert({pos, r, v}).first;
    }

    void setMax(int l, int r) {
        if (l > r)
            return;

        auto itr = split(r + 1);
        auto itl = split(l);

        Int mx = numeric_limits<Int>::min();

        for (auto it = itl; it != itr;) {
```

```
            mx = max(mx, it->v);
            sum -= it->v * (it->r - it->l + 1);
            it = s.erase(it);
        }

        s.insert({l, r, mx});
        sum += mx * (r - l + 1);
    }

    void setMin(int l, int r) {
        if (l > r)
            return;

        auto itr = split(r + 1);
        auto itl = split(l);

        Int mn = numeric_limits<Int>::max();

        for (auto it = itl; it != itr;) {
            mn = min(mn, it->v);
            sum -= it->v * (it->r - it->l + 1);
            it = s.erase(it);
        }

        s.insert({l, r, mn});
        sum += mn * (r - l + 1);
    }
};
```

自动取模

```cpp
template <class T> constexpr T ksm(T a, ll b) {
    T res = 1;
    while (b) {
        if (b & 1)
            res *= a;
        b >>= 1;
        a *= a;
    }
    return res;
}

template <int P> struct MInt {
    int x;
    constexpr MInt() : x() {}
    constexpr MInt(ll x) : x{norm(x % getMod())} {}
    static int Mod;
    constexpr static int getMod() {
        if (P > 0) {
            return P;
        } else {
            return Mod;
        }
    }
    constexpr static void setMod(int Mod_) { Mod = Mod_; }
    constexpr int norm(int x) const {
        if (x < 0) {
            x += getMod();
        }
        if (x >= getMod()) {
            x -= getMod();
        }
        return x;
    }
    constexpr int val() const { return x; }
    explicit constexpr operator int() const { return x; }
    constexpr MInt operator-() const {
        MInt res;
        res.x = norm(getMod() - x);
        return res;
    }
    constexpr MInt inv() const {
        assert(x != 0);
        return ksm(*this, getMod() - 2);
    }
    constexpr MInt &operator*=(MInt rhs) & {
        x = 1LL * x * rhs.x % getMod();
        return *this;
    }
    constexpr MInt &operator+=(MInt rhs) & {
        x = norm(x + rhs.x);
        return *this;
    }
```

```cpp
    constexpr MInt &operator-=(MInt rhs) & {
        x = norm(x - rhs.x);
        return *this;
    }
    constexpr MInt &operator/=(MInt rhs) & { return *this *= rhs.inv(); }
    friend constexpr MInt operator*(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res *= rhs;
        return res;
    }
    friend constexpr MInt operator+(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res += rhs;
        return res;
    }
    friend constexpr MInt operator-(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res -= rhs;
        return res;
    }
    friend constexpr MInt operator/(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res /= rhs;
        return res;
    }
    friend constexpr istream &operator>>(istream &is, MInt &a) {
        ll v;
        is >> v;
        a = MInt(v);
        return is;
    }
    friend constexpr ostream &operator<<(ostream &os, const MInt &a) {
        return os << a.val();
    }
    friend constexpr bool operator==(MInt lhs, MInt rhs) {
        return lhs.val() == rhs.val();
    }
    friend constexpr bool operator!=(MInt lhs, MInt rhs) {
        return lhs.val() != rhs.val();
    }
};

template <> int MInt<0>::Mod = 998244353;

template <int V, int P> constexpr MInt<P> CInv = MInt<P>(V).inv();

using Z = MInt<MOD>;
```